

```
import pandas as pd
import numpy as np
import os, pathlib
from pathlib import Path
```

```
block_df = pd.read_parquet('hblock_df.parquet')
```

```
exp_block_df = compact_to_expanded(
    block_df, timeseries_col='energy_consumption',
    static_cols=['frequency', 'series_length', 'stdorToU', 'Acorn', 'Acorn_grouped'],
    time_varying_cols=['pressure', 'apparentTemperature', 'windSpeed', 'precipType', 'icon', 'humidity', 's
ts_identifier='LCLid')
```

100%799/799 [00:13<00:00, 33.62it/s]

```
exp_block_df.head(1)
```

	timestamp	LCLid	energy_consumption	frequency	series_length	stdorToU	Acorn	Acorn_grouped	pressure
0	2012-10-13	MAC000002	0.263	30min	24144	Std	ACORN-A	Affluent	100

Train, Test and Validation Sets

```
test_mask = (exp_block_df.timestamp.dt.year==2014) & (exp_block_df.timestamp.dt.month==2)
validation_mask = (exp_block_df.timestamp.dt.year==2014) & (exp_block_df.timestamp.dt.month==1)

train = exp_block_df[~(test_mask | validation_mask)]
validation = exp_block_df[validation_mask]
test = exp_block_df[test_mask]
train.shape, validation.shape, test.shape
```

((11855088, 15), (595200, 15), (518400, 15))

Baseline Forecast

```
!pip install statsforecast utilsforecast > /dev/null
!pip install datasetsforecast > /dev/null
```

```
import statsforecast
from functools import partial
from statsforecast.core import StatsForecast
from utilsforecast.plotting import plot_series
from utilsforecast.evaluation import evaluate
from statsforecast.models import (
    Naive, SeasonalNaive, HistoricAverage, WindowAverage, SeasonalWindowAverage,
    RandomWalkWithDrift, HoltWinters, #ETS,
    AutoETS, AutoARIMA, ARIMA, AutoTheta, DynamicTheta, DynamicOptimizedTheta,
    Theta, OptimizedTheta, TBATS, AutoTBATS, MSTL
)
```

```
train_df = train[['LCLid', 'timestamp', 'energy_consumption', 'frequency']]
validation_df = validation[
    ['LCLid', 'timestamp', 'energy_consumption', 'frequency']]
test_df = test[['LCLid', 'timestamp', 'energy_consumption', 'frequency']]
```

```
train_df.timestamp.min(), train_df.timestamp.max(), \
test_df.timestamp.min(), test_df.timestamp.max(), \
validation_df.timestamp.min(), validation_df.timestamp.max()
```

(Timestamp('2012-01-01 00:00:00'),
Timestamp('2013-12-31 23:30:00'),
Timestamp('2014-02-01 00:00:00'),
Timestamp('2014-02-27 23:30:00'),
Timestamp('2014-01-01 00:00:00'),
Timestamp('2014-01-31 23:30:00'))

```
freq_ = train_df.iloc[0]['frequency']
timeseries_train = train_df.loc[train_df.LCLid=='MAC000322', ['LCLid', 'timestamp', 'energy_consumption']]
timeseries_validation = validation_df.loc[validation_df.LCLid=='MAC000322', ['LCLid', 'timestamp', 'energy_
timeseries_test = test_df.loc[test_df.LCLid=='MAC000322', ['LCLid', 'timestamp', 'energy_consumption']]
```

```
predictions = pd.concat([timeseries_train, timeseries_validation])
predictions.head(), predictions.shape, predictions.dtypes
```

```
(      LCLid      timestamp  energy_consumption
0  MAC000322  2012-03-07 00:00:00          0.125
1  MAC000322  2012-03-07 00:30:00          0.104
2  MAC000322  2012-03-07 01:00:00          0.133
3  MAC000322  2012-03-07 01:30:00          0.145
4  MAC000322  2012-03-07 02:00:00          0.109,
(33408, 3),
LCLid      object
timestamp      datetime64[ns]
energy_consumption      float64
dtype: object)
```

Baseline Forecasts - use built-in methods with the package

```
def evaluate_performance(ts_train, ts_test, models, metrics, freq, level, id_col, time_col, target_col, h,
    if metric_df is None:
        metric_df = pd.DataFrame()

    results = ts_test.copy()
    timing = {}

    for model in models:
        model_name = model.__class__.__name__
        evaluation = {}

        start_time = time.time()

        sf = StatsForecast(
            models = [model], freq = freq, n_jobs = 1, fallback_model=Naive()
        )

        y_pred = sf.forecast(
            df = ts_train, h = h, level = level, id_col = id_col, time_col = time_col, target_col = target_
        )

        duration = time.time() - start_time
        timing[model_name] = duration

        results = results.merge(y_pred, how='left', on=[id_col, time_col])

        ids = ts_train[id_col].unique()
        for id in ids:
            temp_results = results[results[id_col] == id]
            temp_train = ts_train[ts_train[id_col] == id]

            for metric in metrics:
                metric_name = metric.__name__
                if metric_name == 'mase':
                    evaluation[metric_name] = metric(temp_results[target_col], temp_results[model_name], te
                else:
                    evaluation[metric_name] = metric(temp_results[target_col], temp_results[model_name])

            evaluation[id_col] = id
            evaluation['Time Elapsed'] = timing[model_name]
            evaluation['Model'] = model_name

            temp_df = pd.DataFrame(evaluation, index=[0])
            temp_df['Model'] = model_name
            metric_df = pd.concat([metric_df, temp_df])

    return results, metric_df
```

```
''' add code to utility for eval-performance '''
import time
```

Naive Forecast

```
metrics = pd.DataFrame()

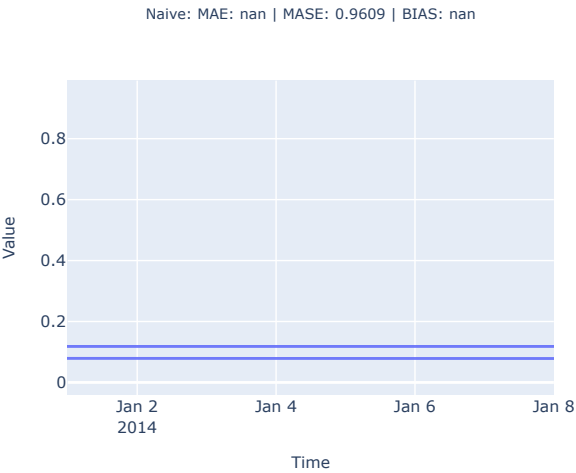
results, metrics = evaluate_performance(
    ts_train=timeseries_train,
    ts_test=timeseries_validation,
    models=[Naive()],
    metrics=[mase, mae, mse, forecast_bias],
    freq=freq_,
    level=[],
    id_col='LCLid',
    time_col='timestamp',
    target_col='energy_consumption',
    h=len(timeseries_validation),
    metric_df=metrics
)
```

```
model_name = ['Naive']
model_display_name = ['Naive']
```

```
fig = plot_forecast(
    results, forecast_columns=model_name,forecast_display_names=model_display_name)

fig = format_plot(fig)
fig.update_layout(title_text=f"{model_name[0]}: "\
                    f"MAE: {metrics.loc[metrics.Model==model_name[0]][['mae']].iloc[0].item():.4f} | "\
                    f"MASE: {metrics.loc[metrics.Model==model_name[0]][['mase']].iloc[0].item():.4f} | "\
                    f"BIAS: {metrics.loc[metrics.Model==model_name[0]][['forecast_bias']].iloc[0].item():.4f}
fig.update_xaxes(type="date", range=["2014-01-01", "2014-01-08"])

fig.show()
```



metrics

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	0.960906	MAC000322	0.125081	Naive	NaN	NaN	NaN
0	0.960906	MAC000322	0.125081	Naive	0.060111	NaN	NaN
0	0.960906	MAC000322	0.125081	Naive	0.060111	0.010746	NaN
0	0.960906	MAC000322	0.125081	Naive	0.060111	0.010746	3.555109

Seasonal Naive Forecast

```
metrics = pd.DataFrame()

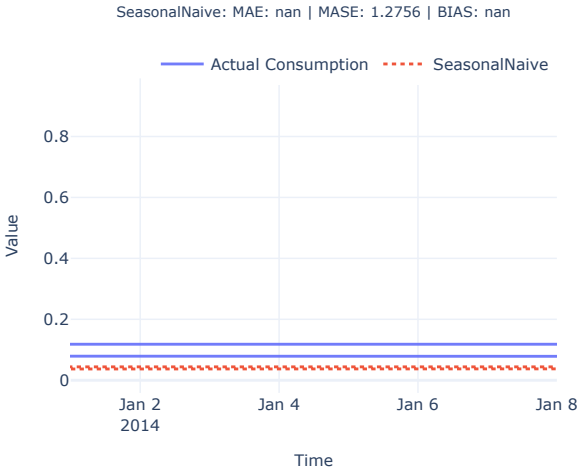
results, metrics = (
    evaluate_performance(
        ts_train=timeseries_train,
        ts_test=timeseries_validation,
        models = [SeasonalNaive(season_length=48*7)],
        metrics = [mase, mae, mse, forecast_bias],
        freq = freq_,
        level = [] ,
        id_col = 'LCLid',
        time_col = 'timestamp',
        target_col = 'energy_consumption',
        h = len(timeseries_validation),
        metric_df=metrics )
)
```

```
model_name = ['SeasonalNaive']
model_display_name = ['SeasonalNaive']

fig = plot_forecast(
    results, forecast_columns=model_name,forecast_display_names=model_display_name)

fig = format_plot(fig)
fig.update_layout(title_text=f"{model_name[0]}: "\
                    f"MAE: {metrics.loc[metrics.Model==model_name[0]][['mae']].iloc[0].item():.4f} | "\
                    f"MASE: {metrics.loc[metrics.Model==model_name[0]][['mase']].iloc[0].item():.4f} | "\
                    f"BIAS: {metrics.loc[metrics.Model==model_name[0]][['forecast_bias']].iloc[0].item():.4f}
fig.update_xaxes(type="date", range=["2014-01-01", "2014-01-08"])

fig.show()
```



metrics

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	1.2756	MAC000322	0.106611	SeasonalNaive	NaN	NaN	NaN
0	1.2756	MAC000322	0.106611	SeasonalNaive	0.079797	NaN	NaN
0	1.2756	MAC000322	0.106611	SeasonalNaive	0.079797	0.017751	NaN
0	1.2756	MAC000322	0.106611	SeasonalNaive	0.079797	0.017751	15.270441

Moving Average Forecast

```
metrics = pd.DataFrame()

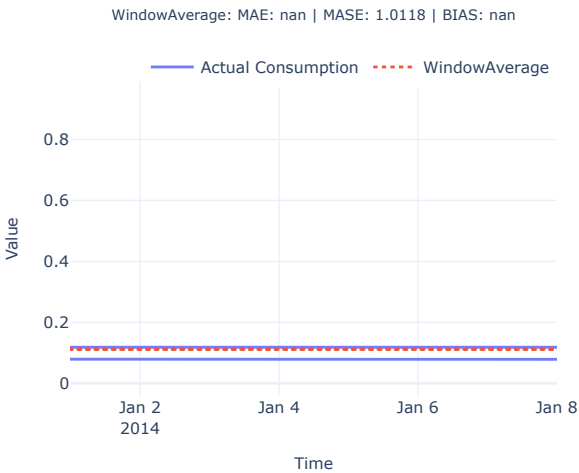
results, metrics = (
    evaluate_performance(
        ts_train=timeseries_train,
        ts_test=timeseries_validation,
        models = [WindowAverage(window_size=48)],
        metrics = [mase, mae, mse, forecast_bias],
        freq = freq_,
        level = [] ,
        id_col = 'LCLid',
        time_col = 'timestamp',
        target_col = 'energy_consumption',
        h = len(timeseries_validation),
        metric_df=metrics )
)
```

```
model_name = ['WindowAverage']
model_display_name = ['WindowAverage']

fig = plot_forecast(
    results, forecast_columns=model_name,forecast_display_names=model_display_name)

fig = format_plot(fig)
fig.update_layout(title_text=f"{model_name[0]}: "\
    f"MAE: {metrics.loc[metrics.Model==model_name[0]][['mae']].iloc[0].item():.4f} | "\
    f"MASE: {metrics.loc[metrics.Model==model_name[0]][['mase']].iloc[0].item():.4f} | "\
    f"BIAS: {metrics.loc[metrics.Model==model_name[0]][['forecast_bias']].iloc[0].item():.4f}")
fig.update_xaxes(type="date", range=["2014-01-01", "2014-01-08"])

fig.show()
```



metrics

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	1.011826	MAC000322	0.088453	WindowAverage	NaN	NaN	NaN
0	1.011826	MAC000322	0.088453	WindowAverage	0.063296	NaN	NaN
0	1.011826	MAC000322	0.088453	WindowAverage	0.063296	0.010798	NaN
0	1.011826	MAC000322	0.088453	WindowAverage	0.063296	0.010798	-7.817351

Exponential Smoothing Forecast

```
metrics = pd.DataFrame()

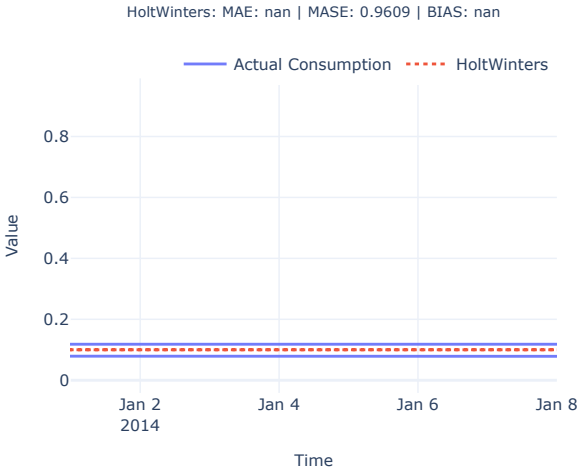
results, metrics = (
    evaluate_performance(
        ts_train=timeseries_train,
        ts_test=timeseries_validation,
        models = [HoltWinters(error_type = 'A', season_length = 48)],
        metrics = [mase, mae, mse, forecast_bias],
        freq = freq_,
        level = [] ,
        id_col = 'LCLid',
        time_col = 'timestamp',
        target_col = 'energy_consumption',
        h = len(timeseries_validation),
        metric_df=metrics )
)
```

```
model_name = ['HoltWinters']
model_display_name = ['HoltWinters']

fig = plot_forecast(
    results, forecast_columns=model_name,forecast_display_names=model_display_name)

fig = format_plot(fig)
fig.update_layout(title_text=f"{model_name[0]}: "\
    f"MAE: {metrics.loc[metrics.Model==model_name[0]][['mae']].iloc[0].item():.4f} | "\
    f"MASE: {metrics.loc[metrics.Model==model_name[0]][['mase']].iloc[0].item():.4f} | "\
    f"BIAS: {metrics.loc[metrics.Model==model_name[0]][['forecast_bias']].iloc[0].item():.4f}")
fig.update_xaxes(type="date", range=["2014-01-01", "2014-01-08"])

fig.show()
```



metrics

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	0.960906	MAC000322	0.159328	HoltWinters	NaN	NaN	NaN
0	0.960906	MAC000322	0.159328	HoltWinters	0.060111	NaN	NaN
0	0.960906	MAC000322	0.159328	HoltWinters	0.060111	0.010746	NaN
0	0.960906	MAC000322	0.159328	HoltWinters	0.060111	0.010746	3.555109

Model AutoETS

```
metrics = pd.DataFrame()

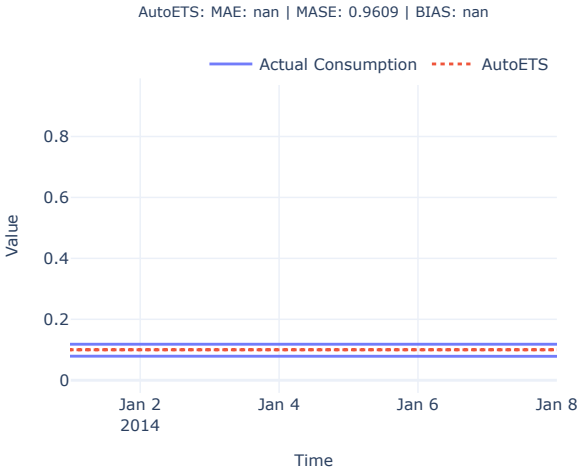
results, metrics = (
    evaluate_performance(
        ts_train=timeseries_train,
        ts_test=timeseries_validation,
        models = [AutoETS(model = 'AAA',season_length = 48)],
        metrics = [mase, mae, mse, forecast_bias],
        freq = freq_,
        level = [] ,
        id_col = 'LCLid',
        time_col = 'timestamp',
        target_col = 'energy_consumption',
        h = len(timeseries_validation),
        metric_df=metrics )
)
```

```
model_name = ['AutoETS']
model_display_name = ['AutoETS']

fig = plot_forecast(
    results, forecast_columns=model_name,forecast_display_names=model_display_name)

fig = format_plot(fig)
fig.update_layout(title_text=f"{model_name[0]}: "\
                    f"MAE: {metrics.loc[metrics.Model==model_name[0]][['mae']].iloc[0].item():.4f} | "\
                    f"MASE: {metrics.loc[metrics.Model==model_name[0]][['mase']].iloc[0].item():.4f} | "\
                    f"BIAS: {metrics.loc[metrics.Model==model_name[0]][['forecast_bias']].iloc[0].item():.4f}")
fig.update_xaxes(type="date", range=["2014-01-01", "2014-01-08"])

fig.show()
```



metrics

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	0.960906	MAC000322	0.100208	AutoETS	NaN	NaN	NaN
0	0.960906	MAC000322	0.100208	AutoETS	0.060111	NaN	NaN
0	0.960906	MAC000322	0.100208	AutoETS	0.060111	0.010746	NaN
0	0.960906	MAC000322	0.100208	AutoETS	0.060111	0.010746	3.555109

ARIMA

```
metrics = pd.DataFrame()

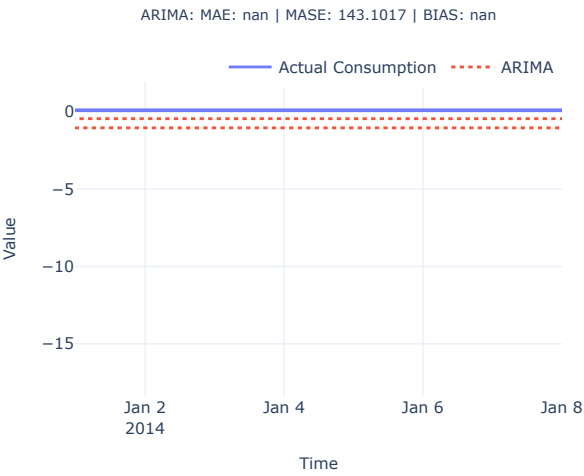
results, metrics = (
    evaluate_performance(
        ts_train=timeseries_train,
        ts_test=timeseries_validation,
        models = [ARIMA(order = (2,1,1), seasonal_order = (1,1,1), season_length = 48)],
        metrics = [mase, mae, mse, forecast_bias],
        freq = freq_,
        level = [] ,
        id_col = 'LCLid',
        time_col = 'timestamp',
        target_col = 'energy_consumption',
        h = len(timeseries_validation),
        metric_df=metrics )
)

model_name = ['ARIMA']
model_display_name = ['ARIMA']

fig = plot_forecast(
    results, forecast_columns=model_name,forecast_display_names=model_display_name)

fig = format_plot(fig)
fig.update_layout(title_text=f"{model_name[0]}: "\
                    f"MAE: {metrics.loc[metrics.Model==model_name[0]][['mae']].iloc[0].item():.4f} | "\
                    f"MASE: {metrics.loc[metrics.Model==model_name[0]][['mase']].iloc[0].item():.4f} | "\
                    f"BIAS: {metrics.loc[metrics.Model==model_name[0]][['forecast_bias']].iloc[0].item():.4f}")
fig.update_xaxes(type="date", range=["2014-01-01", "2014-01-08"])

fig.show()
```



metrics

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	143.101743	MAC000322	116.837616	ARIMA	NaN	NaN	NaN
0	143.101743	MAC000322	116.837616	ARIMA	8.951943	NaN	NaN
0	143.101743	MAC000322	116.837616	ARIMA	8.951943	105.230962	NaN
0	143.101743	MAC000322	116.837616	ARIMA	8.951943	105.230962	8633.644878

▼ Theta

```
metrics = pd.DataFrame()

results, metrics = (
    evaluate_performance(
        ts_train=timeseries_train,
        ts_test=timeseries_validation,
        models = [Theta(season_length =48, decomposition_type = 'additive' )],
        metrics = [mase, mae, mse, forecast_bias],
        freq = freq_,
        level = [] ,
        id_col = 'LCLid',
        time_col = 'timestamp',
        target_col = 'energy_consumption',
        h = len(timeseries_validation),
        metric_df=metrics )
)
```

```
model_name = ['Theta']
model_display_name = ['Theta']

fig = plot_forecast(
    results, forecast_columns=model_name,forecast_display_names=model_display_name)

fig = format_plot(fig)
fig.update_layout(title_text=f"{model_name[0]}: \"
    f\"MAE: {metrics.loc[metrics.Model==model_name[0]][['mae']].iloc[0].item():.4f} | \"
    f\"MASE: {metrics.loc[metrics.Model==model_name[0]][['mase']].iloc[0].item():.4f} | \"
    f\"BIAS: {metrics.loc[metrics.Model==model_name[0]][['forecast_bias']].iloc[0].item():.4f}
fig.update_xaxes(type="date", range=["2014-01-01", "2014-01-08"])

fig.show()
```

rmse, smape

▼ TBATS

```
metrics = pd.DataFrame()

results, metrics = (
    evaluate_performance(
        ts_train=timeseries_train,
        ts_test=timeseries_validation,
        models = [TBATS(season_length = 48, use_boxcox=False)],
        metrics = [mase, mae, mse, forecast_bias],
        freq = freq_,
        level = [] ,
        id_col = 'LCLid',
        time_col = 'timestamp',
```



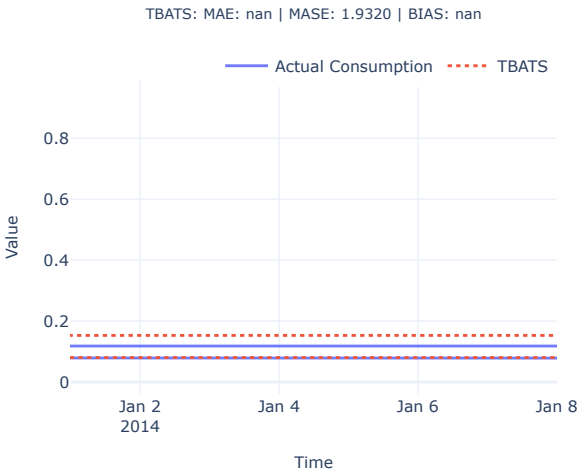
```
target_col = 'energy_consumption',
h = len(timeseries_validation),
metric_df=metrics )
)
```

```
model_name = ['TBATS']
model_display_name = ['TBATS']

fig = plot_forecast(
    results, forecast_columns=model_name,forecast_display_names=model_display_name)

fig = format_plot(fig)
fig.update_layout(title_text=f"{model_name[0]}: "\
                    f"MAE: {metrics.loc[metrics.Model==model_name[0]][['mae']].iloc[0].item():.4f} | "\
                    f"MASE: {metrics.loc[metrics.Model==model_name[0]][['mase']].iloc[0].item():.4f} | "\
                    f"BIAS: {metrics.loc[metrics.Model==model_name[0]][['forecast_bias']].iloc[0].item():.4f}")
fig.update_xaxes(type="date", range=["2014-01-01", "2014-01-08"])

fig.show()
```



metrics

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	1.93201	MAC000322	1.170573	TBATS	NaN	NaN	NaN
0	1.93201	MAC000322	1.170573	TBATS	0.12086	NaN	NaN
0	1.93201	MAC000322	1.170573	TBATS	0.12086	0.020722	NaN
0	1.93201	MAC000322	1.170573	TBATS	0.12086	0.020722	-90.322055

▼ MSTL

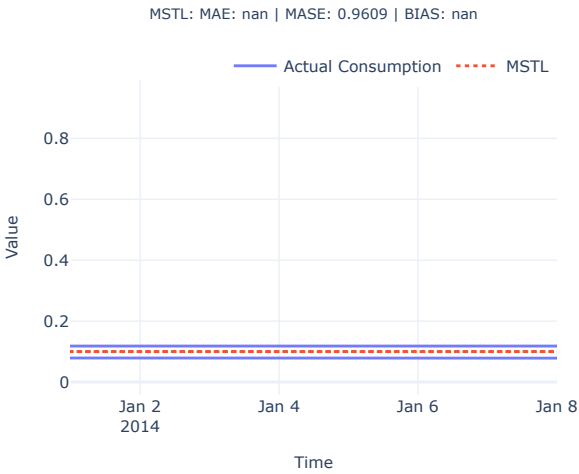
```
metrics = pd.DataFrame()

results, metrics = (
    evaluate_performance(
        ts_train=timeseries_train,
        ts_test=timeseries_validation,
        models = [MSTL(season_length = 48)],
        metrics = [mase, mae, mse, forecast_bias],
        freq = freq_,
        level = [] ,
        id_col = 'LCLid',
        time_col = 'timestamp',
        target_col = 'energy_consumption',
        h = len(timeseries_validation),
        metric_df=metrics )
)
```

```
model_name = ['MSTL']
model_display_name = ['MSTL']

fig = plot_forecast(
    results, forecast_columns=model_name,forecast_display_names=model_display_name)

fig = format_plot(fig)
fig.update_layout(title_text=f"{model_name[0]}: "\
                    f"MAE: {metrics.loc[metrics.Model==model_name[0]][['mae']].iloc[0].item():.4f} | "\
                    f"MASE: {metrics.loc[metrics.Model==model_name[0]][['mase']].iloc[0].item():.4f} | "\
                    f"BIAS: {metrics.loc[metrics.Model==model_name[0]][['forecast_bias']].iloc[0].item():.4f}")
fig.update_xaxes(type="date", range=["2014-01-01", "2014-01-08"])
```



metrics

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	0.960906	MAC000322	0.089476	MSTL	NaN	NaN	NaN
0	0.960906	MAC000322	0.089476	MSTL	0.060111	NaN	NaN
0	0.960906	MAC000322	0.089476	MSTL	0.060111	0.010746	NaN
0	0.960906	MAC000322	0.089476	MSTL	0.060111	0.010746	3.555109

Forecasting for Validation period for several models

```
validation_models = [
    #ARIMA(order = (2,1,1), seasonal_order = (1,1,1), season_length = 48),
    AutoETS(model = 'AAA',season_length = 48),
    TBATS(season_length = 48, use_boxcox=False)
]
validation_models_names = [
    model.__class__.__name__ for model in validation_models]
metric_df = pd.DataFrame([])
h_val = 1488

aggregated_val_metrics = pd.DataFrame()
baseline_val_pred_df, aggregated_val_metrics = (
    evaluate_performance(
        timeseries_train[["LCLid","timestamp","energy_consumption"]],
        timeseries_validation[["LCLid","timestamp","energy_consumption"]],
        models =validation_models,
        metrics = [mase, mae, mse, forecast_bias], # rmse, sshape
        freq = freq_,
        level = [] ,
        id_col = 'LCLid',
        time_col = 'timestamp',
        target_col = 'energy_consumption',
        h = h_val,
        metric_df = aggregated_val_metrics
    )
)
```

aggregated_val_metrics.head()

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	0.960906	MAC000322	0.124938	AutoETS	NaN	NaN	NaN
0	0.960906	MAC000322	0.124938	AutoETS	0.060111	NaN	NaN
0	0.960906	MAC000322	0.124938	AutoETS	0.060111	0.010746	NaN
0	0.960906	MAC000322	0.124938	AutoETS	0.060111	0.010746	3.555109
0	1.932010	MAC000322	1.159333	TBATS	NaN	NaN	NaN

```
baseline_val_pred_df[baseline_val_pred_df.LCLid == 'MAC000322'].head()
```

	LCLid	timestamp	energy_consumption	AutoETS	TBATS
0	MAC000322	2014-01-01 00:00:00	0.056	0.1	0.160797
1	MAC000322	2014-01-01 00:30:00	0.055	0.1	0.162791
2	MAC000322	2014-01-01 01:00:00	0.104	0.1	0.164392
3	MAC000322	2014-01-01 01:30:00	0.039	0.1	0.165264
4	MAC000322	2014-01-01 02:00:00	0.011	0.1	0.164904

```
aggregated_val_metrics[aggregated_val_metrics.Model == 'TBATS'].sort_values  
(by='mase', ascending=True)
```

	mase	LCLid	Time Elapsed	Model	mae	mse	forecast_bias
0	1.93201	MAC000322	1.159333	TBATS	NaN	NaN	NaN
0	1.93201	MAC000322	1.159333	TBATS	0.12086	NaN	NaN
0	1.93201	MAC000322	1.159333	TBATS	0.12086	0.020722	NaN
0	1.93201	MAC000322	1.159333	TBATS	0.12086	0.020722	-90.322055

Decomposing Time Series

```
exp_block_df.head(2)
```

	timestamp	LCLid	energy_consumption	frequency	series_length	stdorToU	Acorn	Acorn_grouped	pressu
0	2012-03-07 00:00:00	MAC000322	0.125	30min	34704	Std	ACORN-D	Affluent	1024
1	2012-03-07 00:30:00	MAC000322	0.104	30min	34704	Std	ACORN-D	Affluent	1024

Fill in missing values

```
ts = SeasonalInterpolation(seasonal_period=48*7).fit_transform(ts_df.energy_consumption.values.reshape(-1,1
```

Outlier Detection

```
res_df = pd.DataFrame(columns=["# of Outliers", "% of Outliers"])
```

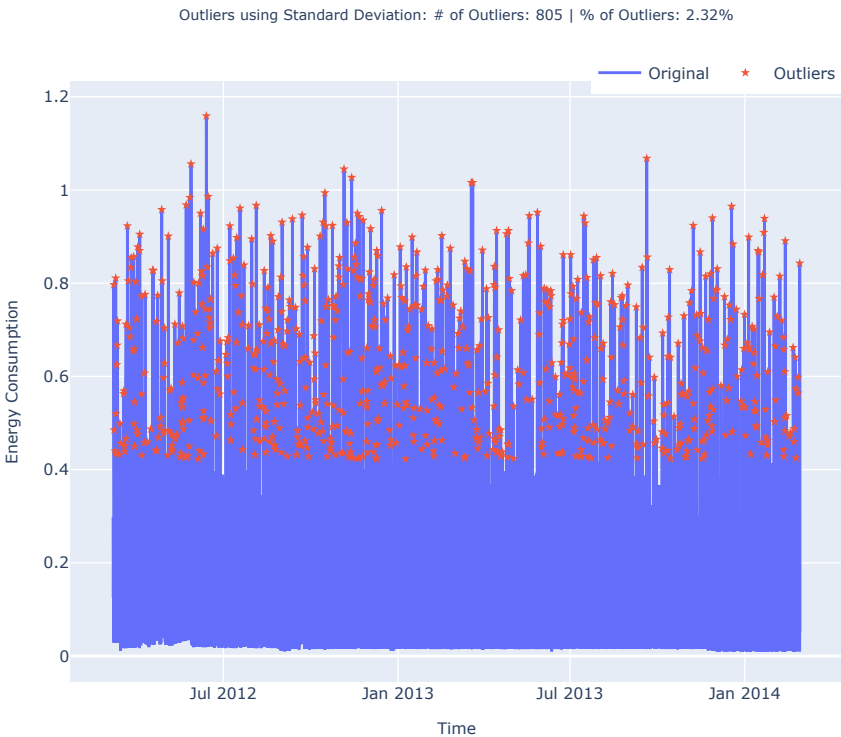
Standard Deviation

```
def detect_outlier_sd(ts, sd_multiple=2):  
    mean = ts.mean()  
    std = ts.std()  
    higher_bound = mean + sd_multiple*std  
    lower_bound = mean - sd_multiple*std  
    outlier_mask = (ts>higher_bound) | (ts<lower_bound)  
    return outlier_mask
```

```
#Detecting Outliers with 3 SD window  
outlier_mask = detect_outlier_sd(ts, sd_multiple=3)  
res_df.loc["3SD", "# of Outliers"] = outlier_mask.sum()  
res_df.loc["3SD", "% of Outliers"] = outlier_mask.sum()/len(ts)*100  
print(f"# of Outliers: {outlier_mask.sum()} | % of Outliers: {outlier_mask.sum()/len(ts)*100:.2f}%")
```

```
# of Outliers: 805 | % of Outliers: 2.32%
```

```
fig = plot_outliers(ts_df.index, ts, outlier_mask, method="Standard Deviation")  
format_plot(fig, xlabel="Time", ylabel="Energy Consumption")  
fig.show()
```

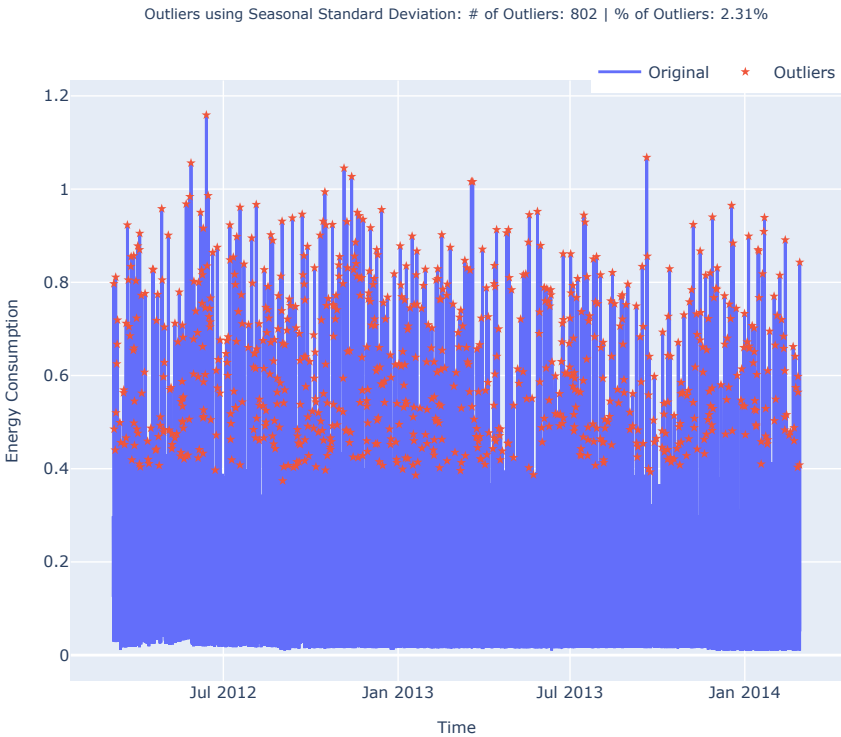


```
stl = MultiSeasonalDecomposition(seasonal_model="fourier",seasonality_periods=["day_of_year", "day_of_week"]
res = stl.fit(pd.Series(ts, index=ts_df.index))
```

```
#Detecting Outliers with 2 SD window
outlier_mask = detect_outlier_sd(res.resid, 3)
res_df.loc["2SD on Residuals", "# of Outliers"] = outlier_mask.sum()
res_df.loc["2SD on Residuals", "% of Outliers"] = outlier_mask.sum()/len(ts)*100
print(f"# of Outliers: {outlier_mask.sum()} | % of Outliers: {outlier_mask.sum()/len(ts)*100:.2f}%")
```

of Outliers: 802 | % of Outliers: 2.31%

```
fig = plot_outliers(ts_df.index, ts, outlier_mask, method="Seasonal Standard Deviation")
format_plot(fig, xlabel="Time", ylabel="Energy Consumption")
fig.show()
```



IQR

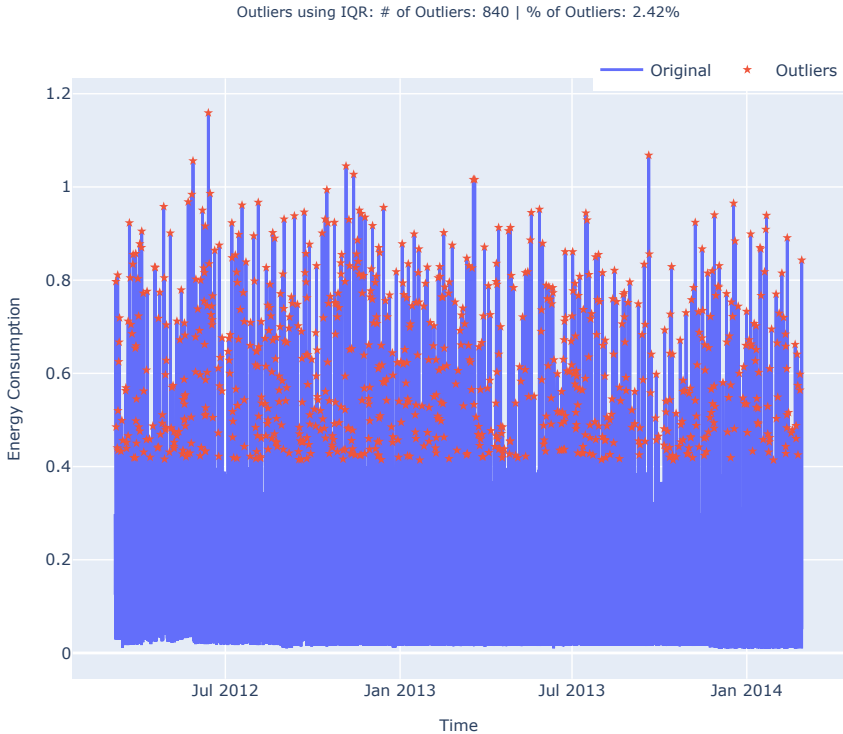
```
def detect_outlier_iqr(ts, iqr_multiple=2):
    q1, q2, q3 = np.quantile(ts, 0.25), np.quantile(ts, 0.5), np.quantile(ts, 0.75)
    iqr = q3-q1
    higher_bound = q3 + iqr_multiple*iqr
```

```
lower_bound = q1 - iqr_multiple*iqr
outlier_mask = (ts>higher_bound) | (ts<lower_bound)
return outlier_mask
```

```
#Detecting Outliers with 4 IQR window
outlier_mask = detect_outlier_iqr(ts, 4)
res_df.loc["4IQR", "# of Outliers"] = outlier_mask.sum()
res_df.loc["4IQR", "% of Outliers"] = outlier_mask.sum()/len(ts)*100
print(f"# of Outliers: {outlier_mask.sum()} | % of Outliers: {outlier_mask.sum()/len(ts)*100:.2f}%")
```

of Outliers: 840 | % of Outliers: 2.42%

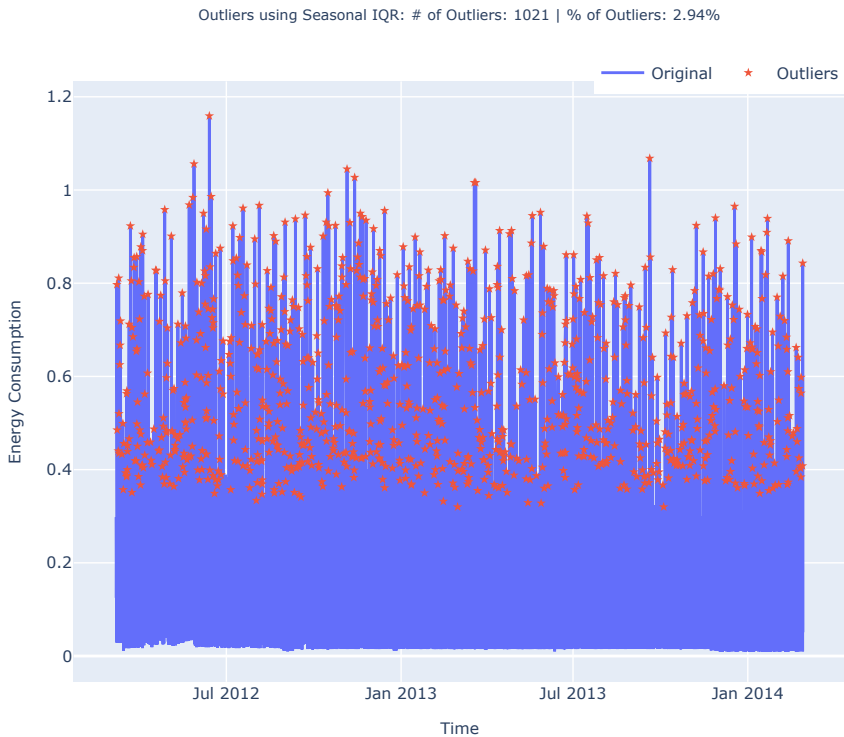
```
fig = plot_outliers(ts_df.index, ts, outlier_mask, method="IQR")
format_plot(fig, xlabel="Time", ylabel="Energy Consumption")
fig.show()
```



```
#Detecting Outliers with 4 IQR window on deseasonalized data
outlier_mask = detect_outlier_iqr(res.resid, 4)
res_df.loc["4SD on Residuals", "# of Outliers"] = outlier_mask.sum()
res_df.loc["4SD on Residuals", "% of Outliers"] = outlier_mask.sum()/len(ts)*100
print(f"# of Outliers: {outlier_mask.sum()} | % of Outliers: {outlier_mask.sum()/len(ts)*100:.2f}%")
```

of Outliers: 1021 | % of Outliers: 2.94%

```
fig = plot_outliers(ts_df.index, ts, outlier_mask, method="Seasonal IQR")
format_plot(fig, xlabel="Time", ylabel="Energy Consumption")
fig.show()
```



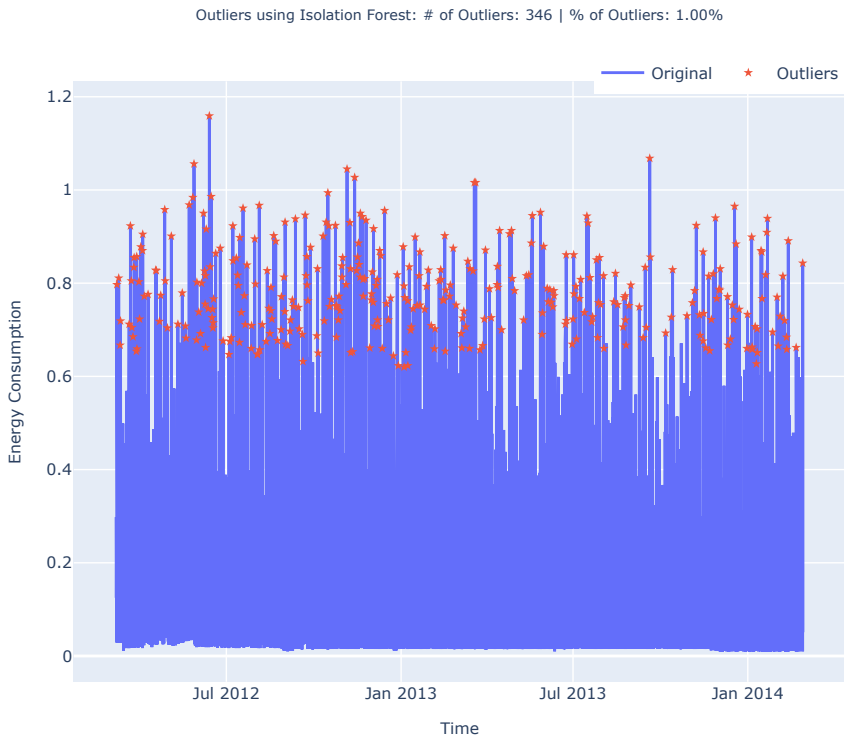
Isolation Forest

```
#class sklearn.ensemble.IsolationForest(*, n_estimators=100, max_samples='auto',
#                                       contamination='auto', max_features=1.0, bootstrap=False, n_jobs=None)
```

```
outlier_mask = detect_outlier_isolation_forest(ts, outlier_fraction=0.01)
res_df.loc["Isolation Forest", "# of Outliers"] = outlier_mask.sum()
res_df.loc["Isolation Forest", "% of Outliers"] = outlier_mask.sum()/len(ts)*100
print(f"# of Outliers: {outlier_mask.sum()} | % of Outliers: {outlier_mask.sum()/len(ts)*100:.2f}%")
```

of Outliers: 346 | % of Outliers: 1.00%

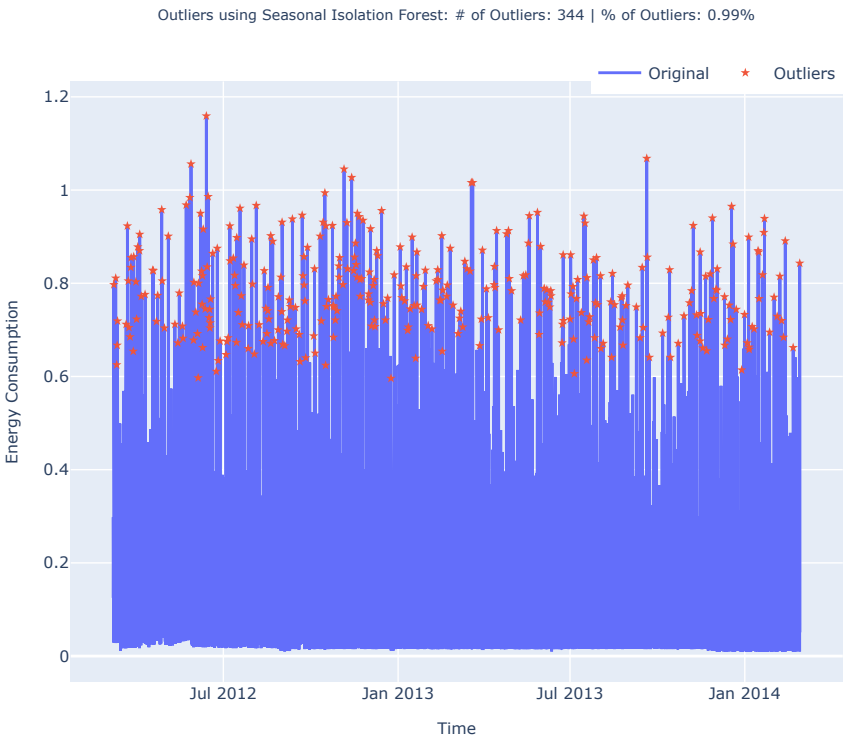
```
fig = plot_outliers(ts_df.index, ts, outlier_mask, method="Isolation Forest")
format_plot(fig, xlabel="Time", ylabel="Energy Consumption")
fig.show()
```



```
outlier_mask = detect_outlier_isolation_forest(res.resid.values, outlier_fraction=0.01)
res_df.loc["Isolation Forest on Residuals", "# of Outliers"] = outlier_mask.sum()
res_df.loc["Isolation Forest on Residuals", "% of Outliers"] = outlier_mask.sum()/len(ts)*100
print(f"# of Outliers: {outlier_mask.sum()} | % of Outliers: {outlier_mask.sum()/len(ts)*100:.2f}%")
```

of Outliers: 344 | % of Outliers: 0.99%

```
fig = plot_outliers(ts_df.index, ts, outlier_mask, method="Seasonal Isolation Forest")
format_plot(fig, xlabel="Time", ylabel="Energy Consumption")
fig.show()
```



Extreme Studentized Deviate (ESD) and Seasonal Extreme Studentized Deviate (S-ESD)

```
outlier_mask = generalized_esd(ts, max_anomalies=800, alpha=0.05, hybrid=False)
res_df.loc["ESD", "# of Outliers"] = outlier_mask.sum()
res_df.loc["ESD", "% of Outliers"] = outlier_mask.sum()/len(ts)*100
print(f"# of Outliers: {outlier_mask.sum()} | % of Outliers: {outlier_mask.sum()/len(ts)*100:.2f}%")
```

of Outliers: 800 | % of Outliers: 2.31%

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.